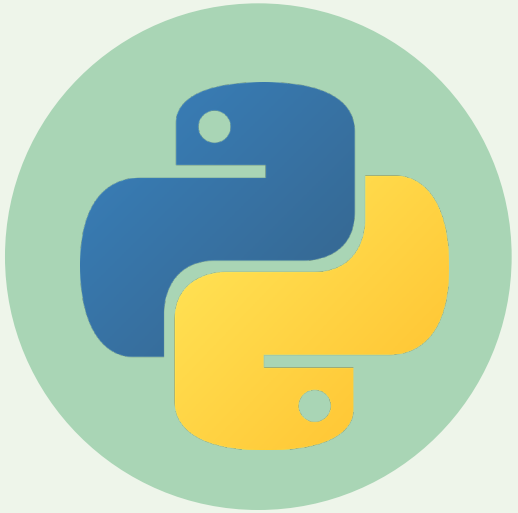
PYTHON · OOP

PYTHON FUNDAMENTALS FOR OOP



9 Groups | Code + Output | Java vs Python

Name: HIN Panha
ID: e20241363
Group: I3-AMS-1A

Course: Object-Oriented Programming (OOP)
Lecturer: VANN Visal

GROUP 1 OF 9

PYTHON BASIC DATA TYPES

TOPICS IN THIS GROUP

1. Primitive Types
2. Collections: list and tuple
3. Collections: dict, set and objects
4. Literals

★ **PRACTICAL EXAMPLE** · Student Records

PRIMITIVE TYPES

1. int: whole numbers of any size, such as 20 or -3.
2. float: decimal numbers such as 85.5; used for scores and averages.
3. bool: only True or False; used in conditions.
4. str: text inside quotes; immutable, so a change creates a new string.

CODE

```
age = 20
score = 85.5
passed = True
name = "Dara"
print(type(age), type(score))
print(type(passed), type(name))
```

OUTPUT

```
<class 'int'> <class 'float'>
<class 'bool'> <class 'str'>
```

COLLECTIONS: LIST AND TUPLE

1. list [ ] keeps items in order and can be changed.
2. tuple ( ) keeps items in order but cannot be changed after creation.
3. Both use an index that starts at 0.
4. Use a list for scores that grow; a tuple for fixed data such as (name, age).

CODE

```
scores = [85.5, 90, 72]
scores.append(88)
info = ("Dara", 20)
print(scores[0], len(scores))
print(info[1])
# info[1] = 21 -> TypeError
```

OUTPUT

```
85.5 4
20
```

COLLECTIONS: DICT, SET AND OBJECTS

1. dict stores key-value pairs; you look up a value by its key.
2. set stores unique items only; duplicates are removed.
3. An object is an instance of a class; a variable holds a reference to it.
4. Two variables can refer to the same object, so a change shows in both.

CODE

```
student = {"name": "Dara", "age": 20}
print(student["name"])
ids = {101, 102, 101}
print(ids)
a = [1, 2]
b = a
b.append(3)
print(a)
```

OUTPUT

```
Dara
{101, 102}
[1, 2, 3]
```

LITERALS

1. A literal is a value written directly in the code.
2. Numeric: 10, 3.14, 1e3. String: 'A', "text", """multi-line""".
3. Boolean: True and False. Special: None means "no value".
4. Collection: [1, 2], (1, 2), {"a": 1}, {1, 2}.

CODE

```
x = 10          # int literal
y = 3.14        # float literal
s = 'Hello'     # string literal
ok = True       # Boolean literal
nothing = None  # None literal
items = [1, 2]  # list literal
d = {"a": 1}    # dict literal
```

PRACTICAL EXAMPLE: STUDENT RECORDS

1. Name is a str, age an int, score a float.
2. Each student record is a dict with those values.
3. All records are stored together in one list.
4. A for loop reads each record in turn.

CODE

```
students = [
    {"name": "Dara", "age": 20, "score": 85.5},
    {"name": "Sok", "age": 21, "score": 72.0},
]
for s in students:
    print(s["name"], s["score"])
```

OUTPUT

```
Dara 85.5
Sok 72.0
```

GROUP 2 OF 9

PYTHON VARIABLES AND ATTRIBUTES

TOPICS IN THIS GROUP

1. Local Variables
2. Instance Attributes
3. Class Attributes
4. Instance vs Class Attribute

★ **PRACTICAL EXAMPLE** · Student Class

LOCAL VARIABLES

1. Created inside a method or function and used only there.
2. Disappears when the method finishes.
3. Not stored with self, so other methods cannot see it.
4. Use it for temporary calculations.

CODE

```
class Student:
    def average(self, a, b):
        total = a + b      # local
        return total / 2
s = Student()
print(s.average(80, 90))
# print(total) -> NameError
```

OUTPUT

```
85.0
```

INSTANCE ATTRIBUTES

1. Created with self.name = value, usually in __init__.
2. Every object keeps its own copy.
3. Changing one object does not affect the others.
4. Read with object.name.

CODE

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
a = Student("Dara", 20)
b = Student("Sok", 21)
print(a.name, b.name)
```

OUTPUT

```
Dara Sok
```

CLASS ATTRIBUTES

1. Defined in the class body, outside any method.
2. One copy is shared by all instances.
3. Read with Student.school or obj.school.
4. Student.school = … updates it for every student.

CODE

```
class Student:
    school = "Royal School"
a = Student()
b = Student()
print(a.school, b.school)
Student.school = "ABC School"
print(a.school, b.school)
```

OUTPUT

```
Royal School Royal School
ABC School ABC School
```

INSTANCE VS CLASS ATTRIBUTE

1. a.school = "X" creates a new instance attribute only on a.
2. The class attribute and other objects stay unchanged.
3. Python looks in the object first, then in the class.
4. Use class attributes for shared data, instance attributes for personal data.

CODE

```
class Student:
    school = "Royal School"
a = Student()
b = Student()
a.school = "Private School"
print(a.school, b.school)
```

OUTPUT

```
Private School Royal School
```

PRACTICAL EXAMPLE: STUDENT CLASS

1. school is a class attribute shared by every student.
2. name is an instance attribute, different for each student.
3. total is a class attribute that counts how many students exist.
4. Each new object adds 1 to Student.total.

CODE

```
class Student:
    school = "Royal School"
    total = 0
    def __init__(self, name):
        self.name = name
        Student.total += 1
a = Student("Dara")
b = Student("Sok")
print(a.name, b.name, Student.school)
print(Student.total)
```

OUTPUT

```
Dara Sok Royal School
2
```

GROUP 3 OF 9

MODIFIERS AND ACCESS CONTROL, PART 1

TOPICS IN THIS GROUP

1. Public, Non-Public and Private Names
2. Access Control and Name Mangling
3. The @staticmethod Decorator
4. The @classmethod Decorator

★ **PRACTICAL EXAMPLE** · BankAccount

PUBLIC, NON-PUBLIC AND PRIVATE NAMES

1. Python has no real private keyword; it uses naming rules.
2. name: public, free to use anywhere.
3. _name: non-public, a hint that it is for internal use.
4. __name: private, Python renames it (name mangling).

CODE

```
class Account:
    def __init__(self):
        self.owner = "Dara"     # public
        self._branch = "Main"   # non-public
        self.__pin = 1234       # private
a = Account()
print(a.owner, a._branch)
```

OUTPUT

```
Dara Main
```

ACCESS CONTROL AND NAME MANGLING

1. Inside the class, __pin is renamed to _Account__pin.
2. Writing a.__pin from outside raises AttributeError.
3. It is still reachable as a._Account__pin: Python trusts the programmer.
4. So access control is by convention; use methods to read or change private data.

CODE

```
class Account:
    def __init__(self):
        self.__pin = 1234
a = Account()
print(a._Account__pin)
print(a.__pin)
```

OUTPUT

```
1234
AttributeError: 'Account' object has no attribute '__pin'
```

THE @STATICMETHOD DECORATOR

1. Declared with @staticmethod; it takes no self or cls.
2. It cannot read object or class data.
3. It groups a related utility function inside the class.
4. Call it with ClassName.method().

CODE

```
class MathTool:
    @staticmethod
    def is_even(n):
        return n % 2 == 0
print(MathTool.is_even(4))
print(MathTool.is_even(7))
```

OUTPUT

```
True
False
```

THE @CLASSMETHOD DECORATOR

1. Declared with @classmethod; the first parameter is cls, the class itself.
2. It can read and change class attributes.
3. Often used as an alternative constructor.
4. It also works when called from a subclass.

CODE

```
class Student:
    school = "Royal School"
    def __init__(self, name):
        self.name = name
    @classmethod
    def from_text(cls, text):
        return cls(text.title())
    @classmethod
    def get_school(cls):
        return cls.school
s = Student.from_text("dara")
print(s.name, Student.get_school())
```

OUTPUT

```
Dara Royal School
```

PRACTICAL EXAMPLE: BANKACCOUNT

1. The balance is private (__balance) and changes only through methods.
2. deposit checks the amount with the static method is_valid.
3. The class method bank_name returns shared class data.

CODE

```
class BankAccount:
    bank = "ABCBank"
    def __init__(self,owner, balance):
        self.owner= owner
        self.__balance= balance
    @staticmethod
    def is_valid(amount):
        returnamount > 0
    def deposit(self,amount):
        if self.is_valid(amount):
            self.__balance+= amount
    def get_balance(self):
        returnself.__balance
    @classmethod
    def bank_name(cls):
        returncls.bank
acc = BankAccount("Dara",100)
acc.deposit(50)
print(acc.get_balance(),BankAccount.bank_name())
```

OUTPUT

```
150 ABC Bank
```

GROUP 4 OF 9

MODIFIERS AND OOP FEATURES, PART 2

TOPICS IN THIS GROUP

1. Abstract Classes and Methods
2. Access Control and Encapsulation
3. The @property Decorator
4. @final and typing.final

★ **PRACTICAL EXAMPLE** · Shape

ABSTRACT CLASSES AND METHODS

1. An abstract class is a template; you cannot create objects from it directly.
2. Inherit from ABC (module abc) to make one.
3. @abstractmethod marks a method that every subclass must override.
4. A subclass that skips it cannot be instantiated.

CODE

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
s = Shape()
```

OUTPUT

```
TypeError: Can't instantiate abstract class Shape
```

ACCESS CONTROL AND ENCAPSULATION

1. Encapsulation puts data and the methods that use it in one class.
2. Hide internal data with _x or __x.
3. Expose controlled access through methods.
4. This protects the data from invalid values.

CODE

```
class Student:
    def __init__(self, score):
        self.__score = score
    def set_score(self, value):
        if 0 <= value <= 100:
            self.__score = value
    def get_score(self):
        return self.__score
s = Student(70)
s.set_score(150)
print(s.get_score())
```

OUTPUT

```
70
```

THE @PROPERTY DECORATOR

1. @property turns a method into an attribute you read without parentheses.
2. Add @score.setter to allow updates with validation.
3. Same safety as getters and setters, with cleaner syntax.
4. Callers write s.score, not s.get_score().

CODE

```
class Student:
    def __init__(self, score):
        self._score = score
    @property
    def score(self):
        return self._score
    @score.setter
    def score(self, v):
        if 0 <= v <= 100:
            self._score = v
s = Student(70)
s.score = 90
print(s.score)
```

OUTPUT

```
90
```

@FINAL AND TYPING.FINAL

1. typing.final marks a method or class that must not be overridden or subclassed.
2. It is a hint for type checkers such as mypy.
3. Python does not enforce it at runtime.
4. In Java, final is enforced by the compiler.

CODE

```
from typing import final
class Base:
    @final
    def show(self):
        print("Fixed")
class Child(Base):
    def show(self):   # mypy reports an error
        print("Changed")
Child().show()
```

OUTPUT

```
Changed
```

PRACTICAL EXAMPLE: SHAPE

1. Shape sets the rule: every shape must have area().
2. Circle and Rectangle each implement it differently.
3. The loop calls area() without caring about the type (polymorphism).
4. Shape() itself can never be created.

CODE

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass
class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r ** 2
class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h
for s in (Circle(2), Rectangle(3, 4)):
    print(s.area())
```

OUTPUT

```
12.56
12
```

GROUP 5 OF 9

PYTHON OPERATORS, PART 1

TOPICS IN THIS GROUP

1. Arithmetic Operators
2. Comparison Operators
3. Bitwise Operators
4. Logical Operators

★ **PRACTICAL EXAMPLE** · Grades and Eligibility

GROUP 5 · PYTHON OPERATORS, PART 1

ARITHMETIC OPERATORS

1. +, -, * add, subtract and multiply.
2. / is true division and always gives a float; // drops the decimals.
3. % gives the remainder; ** is power.
4. Mixing int and float gives a float.

CODE

```
print(7 + 2, 7 - 2, 7 * 2)
print(7 / 2, 7 // 2)
print(7 % 2, 7 ** 2)
```

OUTPUT

```
9 5 14
3.5 3
1 49
```

COMPARISON OPERATORS

1. A comparison compares two values; the result is a bool.
2. == equal, != not equal.
3. >, <, >=, <= compare order.
4. Chaining is allowed: 0 <= score <= 100. Do not confuse == (compare) with = (assign).

CODE

```
score = 78
print(score == 78, score != 78)
print(score > 50, score <= 50)
print(0 <= score <= 100)
```

OUTPUT

```
True False
True False
True
```

BITWISE OPERATORS

1. They work on the binary bits of integers.
2. & AND, | OR, ^ XOR, ~ NOT.
3. << shifts bits left (times 2), >> shifts right (divide by 2).
4. Here 5 is 101 and 3 is 011 in binary.

CODE

```
print(5 & 3)
print(5 | 3)
print(5 ^ 3)
print(~5)
print(5 << 1, 5 >> 1)
```

OUTPUT

```
1
7
6
-6
10 2
```

LOGICAL OPERATORS

1. and: True only if both conditions are True.
2. or: True if at least one condition is True.
3. not: reverses the value.
4. Short-circuit: Python stops as soon as the result is known.

CODE

```
age, score = 20, 78
print(age >= 18 and score >= 50)
print(age < 18 or score >= 50)
print(not (score >= 50))
```

OUTPUT

```
True
True
False
```

PRACTICAL EXAMPLE: GRADES AND ELIGIBILITY

1. The average uses + and /.
2. A comparison checks the passing mark.
3. and combines the score rule and the attendance rule.
4. The result is stored as a bool.

CODE

```
math, python = 80, 70
average = (math + python) / 2
attendance = 90
eligible = average >= 50 and attendance >= 75
print(average, eligible)
```

OUTPUT

```
75.0 True
```

GROUP 6 OF 9

PYTHON OPERATORS, PART 2

TOPICS IN THIS GROUP

1. Assignment Operators
2. Identity and Membership Operators
3. Conditional Expressions
4. Operator Precedence and Associativity

★ **PRACTICAL EXAMPLE** · Updating and Checking Data

GROUP 6 · PYTHON OPERATORS, PART 2

ASSIGNMENT OPERATORS

1. = stores a value in a variable.
2. Compound operators update in place: x += 5 means x = x + 5.
3. Others: -=, *=, /=, //=, %=, **=.
4. They also work on attributes, such as self.score += 5.

CODE

```
x = 10
x += 5
x *= 2
x -= 3
print(x)
x //= 4
print(x)
```

OUTPUT

```
27
6
```

IDENTITY AND MEMBERSHIP OPERATORS

1. is checks if two names point to the same object; == checks equal values.
2. is not is the opposite of is.
3. in checks whether an item is inside a list, string, set or dict keys.
4. not in is the opposite of in.

CODE

```
a = [1, 2]
b = [1, 2]
c = a
print(a == b, a is b, a is c)
print(2 in a, 5 not in a)
print("Py" in "Python")
```

OUTPUT

```
True False True
True True
True
```

CONDITIONAL EXPRESSIONS

1. Syntax: value_if_true if condition else value_if_false.
2. Shorter than a full if-else for a simple choice.
3. Java equivalent: condition ? a : b.
4. Keep it simple; avoid long nested ones.

CODE

```
score = 72
status = "Pass" if score >= 50 else "Fail"
print(status)
grade = "A" if score >= 80 else "B"
print(grade)
```

OUTPUT

```
Pass
B
```

GROUP 6 · PYTHON OPERATORS, PART 2

OPERATOR PRECEDENCE AND ASSOCIATIVITY

1. Precedence decides which operator runs first.
2. High to low: **, * / // %, + -, comparisons, not, and, or.
3. Associativity: equal operators go left to right; ** goes right to left.
4. Use parentheses to make your intent clear.

CODE

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 ** 2)
print(10 - 4 - 3)
print(not True or True)
```

OUTPUT

```
14
20
512
3
True
```

PRACTICAL EXAMPLE: UPDATING AND CHECKING DATA

1. += updates an object attribute.
2. in checks membership in a list.
3. and joins two conditions.
4. A conditional expression picks the label.

CODE

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
subjects = ["Math", "Python"]
s = Student("Dara", 70)
s.score += 10
if "Python" in subjects and s.score >= 50:
    status = "Excellent" if s.score >= 80 else "Good"

    print(s.name, s.score, status)
```

OUTPUT

```
Dara 80 Excellent
```

GROUP 7 OF 9

PYTHON DECISION-MAKING STATEMENTS

TOPICS IN THIS GROUP

1. The if Statement
2. The if-else Statement
3. The if-elif-else Statement

★ **PRACTICAL EXAMPLE** · Pass/Fail and Letter Grades

THE IF STATEMENT

1. Runs the indented block only if the condition is True.
2. The if line ends with a colon :.
3. Indentation (4 spaces) marks the block.
4. If the condition is False, Python skips the block.

CODE

```
score = 72
if score >= 50:
    print("You passed")
print("Done")
```

OUTPUT

```
You passed
Done
```

THE IF-ELSE STATEMENT

1. else handles the case when the condition is False.
2. Exactly one of the two blocks runs.
3. else has no condition of its own.

CODE

```
score = 40
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

OUTPUT

```
Fail
```

THE IF-ELIF-ELSE STATEMENT

1. Use it when there are more than two outcomes.
2. Python checks from top to bottom and runs the first True block.
3. elif can be repeated; else is the optional default.
4. Order matters: test the highest range first.

CODE

```
score = 72
if score >= 80:
    grade = "A"
elif score >= 70:
    grade = "B"
elif score >= 50:
    grade = "C"
else:
    grade = "F"
print(grade)
```

OUTPUT

```
B
```

PRACTICAL EXAMPLE: PASS/FAIL AND LETTER GRADES

1. if-else decides pass or fail.
2. if-elif-else assigns the letter grade.
3. The loop repeats both decisions for three scores.

CODE

```
for score in (85, 72, 45):
    if score >= 50:
        status = "Pass"
    else:
        status = "Fail"
    if score >= 80:
        grade = "A"
    elif score >= 70:
        grade = "B"
    else:
        grade = "F"
    print(score, status, grade)
```

OUTPUT

```
85 Pass A
72 Pass B
45 Fail F
```

GROUP 8 OF 9

PYTHON DECISION-MAKING, PART 2

TOPICS IN THIS GROUP

1. Nested if Statements
2. Multiple Branches with match-case
3. More match-case Patterns
4. Conditional Expression: Java vs Python

★ **PRACTICAL EXAMPLE** · Student Menu System

NESTED IF STATEMENTS

1. A nested if is an if placed inside another if.
2. The inner condition is checked only when the outer one is True.
3. Good for dependent checks, such as age first, then score.
4. Very deep nesting is hard to read; use and when possible.

CODE

```
age, score = 20, 72
if age >= 18:
    if score >= 50:
        print("Eligible")
    else:
        print("Low score")
else:
    print("Too young")
```

OUTPUT

```
Eligible
```

MULTIPLE BRANCHES WITH MATCH-CASE

1. match compares one value against several case patterns (Python 3.10+).
2. The first matching case runs; no break is needed.
3. case _ is the default branch.
4. It replaces long if-elif-else chains.

CODE

```
day = 3
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case _:
        print("Other")
```

OUTPUT

```
Wednesday
```

GROUP 8 · PYTHON DECISION-MAKING, PART 2

MORE MATCH-CASE PATTERNS

1. Combine values with |: case "add" | "new":.
2. Add a guard condition: case n if n > 90:.
3. Match sequences: case (x, y):.
4. Before Python 3.10, use if-elif-else instead.

CODE

```
cmd = "exit"
match cmd:
    case "add" | "new":
        print("Adding")
    case "exit" | "quit":
        print("Bye")
    case _:
        print("Unknown")
```

OUTPUT

```
Bye
```

CONDITIONAL EXPRESSION: JAVA VS PYTHON

1. Java: condition ? value1 : value2.
2. Python: value1 if condition else value2.
3. The condition sits in the middle in Python.
4. It returns a value, so you can assign or print it directly.

CODE

```
age = 20
msg = "Adult" if age >= 18 else "Minor"
print(msg)
print("Even" if age % 2 == 0 else "Odd")
```

OUTPUT

```
Adult
Even
```

PRACTICAL EXAMPLE: STUDENT MENU SYSTEM

1. while True keeps the menu running.
2. match selects the action.
3. A nested if checks that the name is not empty.
4. break leaves the loop on option 3.

CODE

```
students = []
while True:
    print("1.Add 2.Show 3.Exit")
    choice = input("Choose: ")
    match choice:
        case "1":
            name = input("Name: ")
            if name != "":
                students.append(name)
        case "2":
            print(students)
        case "3":
            break
        case _:
            print("Invalid")
```

OUTPUT

```
1.Add 2.Show 3.Exit
Choose: 2
['Dara']
```

GROUP 9 OF 9

PYTHON LOOPS AND LOOP CONTROL

TOPICS IN THIS GROUP

1. The while Loop
2. The for Loop
3. Repetition: for vs while
4. Loop Control: break, continue and pass

★ **PRACTICAL EXAMPLE** · Averages and Searching

THE WHILE LOOP

1. Repeats as long as the condition is True.
2. The condition is checked before every round.
3. Change the variable inside the loop, or it never ends.
4. Best when the number of rounds is unknown.

CODE

```
count = 1
while count <= 3:
    print("Round", count)
    count += 1
```

OUTPUT

```
Round 1
Round 2
Round 3
```

THE FOR LOOP

1. Goes through each item of a sequence: a list, string or range().
2. There is no counter to manage by hand.
3. range(1, 4) gives 1, 2, 3; the end value is excluded.
4. enumerate() gives the index and the item together.

CODE

```
for n in range(1, 4):
    print(n)
for ch in "Py":
    print(ch)
names = ["Dara", "Sok"]
for i, name in enumerate(names):
    print(i, name)
```

OUTPUT

```
1
2
3
P
y
0 Dara
1 Sok
```

REPETITION: FOR VS WHILE

1. for: use it when you know the items or the number of rounds.
2. while: use it when you repeat until a condition changes.
3. Python has no do-while loop.
4. Simulate it with while True and break, so the body runs at least once.

CODE

```
while True:
    n = int(input("Enter n > 0: "))
    if n > 0:
        break
print("Accepted", n)
```

OUTPUT

```
Enter n > 0: -2
Enter n > 0: 5
Accepted 5
```

LOOP CONTROL: BREAK, CONTINUE AND PASS

1. break stops the whole loop immediately.
2. continue skips the rest of this round and starts the next.
3. pass does nothing; it is a placeholder for empty code.
4. break and continue usually sit inside an if.

CODE

```
for n in range(1, 8):
    if n == 2:
        continue
    if n == 5:
        break
    print(n)
```

OUTPUT

```
1
3
4
```

PRACTICAL EXAMPLE: AVERAGES AND SEARCHING

1. A for loop visits each student object.
2. A running total gives the average.
3. A second loop searches by name.
4. break stops the search once the student is found.

CODE

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
students = [Student("Dara", 80), Student("Sok", 65)]

total = 0
for s in students:
    total += s.score
print(total / len(students))
for s in students:
    if s.name == "Sok":
        print("Found", s.score)
        break
```

OUTPUT

```
72.5
Found 65
```

SUMMARY · JAVA VS PYTHON

DIFFERENCES IN SYNTAX AND TYPES

Topic	Java	Python
Typing	Static: int x = 5;	Dynamic: x = 5
Blocks	{}	Indentation and :
Line end	; required	New line, no ;
Logical operators	&& || !	and or not
Current object	this	self
Constructor	Same name as the class	__init__
Comments	// and /* */	#

SUMMARY · JAVA VS PYTHON

DIFFERENCES IN OOP AND CONTROL FLOW

Topic	Java	Python
Access control	public protected private	_x and __x by convention
Abstract class	abstract keyword	ABC + @abstractmethod
Final	final enforced by compiler	typing.final is a hint only
Conditional	c?a:b	a if c else b
Multi-branch	switch	match-case (3.10+)